

AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

Laboratory Report

Lab 2: Merging Login/Register and Employee CRUD into One Application

Project: 24-57685-2_CompanyApp

Student Name	Sams Ridwan Simran
Student ID	24-57685-2
Course	OOP 2 (Object-Oriented Programming 2)
Faculty	Dr. Iftekhar Mobin
University	American International University Bangladesh (AIUB)
Project	24-57685-2_CompanyApp
Lab	Lab 2: Merging Login/Register and Employee CRUD into One App

Course: **OOP 2** | Faculty: **Dr. Iftekhar Mobin**

This report documents the merge of two independent C# Windows Forms applications (Login-and-Register and EmployeeDetails) into a single, buildable application with one SQL Server database, parameterized queries, and a complete login → dashboard → employee CRUD → logout flow.

1. Introduction

This report documents the merge of two independent C# Windows Forms applications — **Login-and-Register** (namespace `Login_and_Register`, .NET Framework 4.7.2, MS Access via `System.Data.OleDb`) and **EmployeeDetails** (namespace `EmployeeDetails`, .NET Framework 4.8, SQL Server via `System.Data.SqlClient`) — into a single application, **24-57685-2_CompanyApp**, built on `EmployeeDetails` as the host project.

2. Objectives

- Merge two separate WinForms projects into one buildable, runnable application with a single executable.
- Identify and resolve six specific conflicts between the two source projects.
- Replace MS Access/OleDb entirely with a single SQL Server database accessed via parameterized `SqlClient` queries.
- Migrate the real user accounts out of the Access database, not invented data.
- Add attribution (`CreatedBy`) to employee records, tied to the logged-in user.
- Produce a clean login → dashboard → employee CRUD → logout → login flow with no orphaned processes.

3. The Six Conflicts

Six structural and technical conflicts existed between the two source projects. Each was identified, traced to specific files/lines, and resolved as summarized below.

#	Conflict	Resolution
1	Different namespaces	All imported forms changed to namespace <code>EmployeeDetails</code> (host project root namespace retained).
2	Different data providers	Removed all <code>OleDb</code> ; new <code>User.cs</code> written with <code>SqlClient</code> + parameterized queries matching <code>Employee.cs</code> style.
3	Two separate databases	Created single SQL Server database <code>dbCompanyApp</code> ; migrated real Access accounts into it.
4	Different framework versions	Targeted .NET Framework 4.8 throughout (matching host project).
5	Two <code>Program.cs</code> / <code>Main()</code>	Discarded Login project <code>Program.cs</code> ; single entry point now starts at <code>frmLogin</code> .
6	Hidden dependency on <code>db_users.mdb</code>	Removed <code>.mdb</code> dependency after migrating its data into SQL Server.

3.1 Different namespaces. `Login-and-Register` used `Login_and_Register`; `EmployeeDetails` used `EmployeeDetails`. Fixed by changing the namespace of every imported form (`.cs` and `.Designer.cs`) to `EmployeeDetails`, per the host-project requirement that the root namespace stay `EmployeeDetails`.

3.2 Different data providers. `Login-and-Register` used `System.Data.OleDb`; `EmployeeDetails` used `System.Data.SqlClient`. Fixed by removing all `OleDb` code and writing a new `User.cs` class in the same `SqlClient` / parameterized-query style as the existing `Employee.cs`.

3.3 Two separate databases. An Access file (`db_users.mdb`) and a SQL Server database (`dbEmployeeDetails`) existed side by side with no relationship. Fixed by creating one SQL Server database, `dbCompanyApp`, and migrating the real Access accounts into it.

3.4 Different framework versions. Login-and-Register targeted .NET Framework 4.7.2; EmployeeDetails targeted 4.8. Fixed by targeting 4.8 throughout the merged project.

3.5 Two Program.cs / Main() methods. Each project had its own entry point. Fixed by discarding the Login project's Program.cs entirely and keeping exactly one, which now starts the application at frmLogin.

3.6 Hidden dependency on db_users.mdb. The Access file was only present under bin\Debug, outside the tracked project structure — a hidden runtime dependency. Fixed by removing the .mdb dependency altogether once its data was migrated into SQL Server.

4. Database Design

One database, **dbCompanyApp**, holds two related tables.

4.1 dbo.Users

Column	Type	Constraints / Notes
UserID	INT IDENTITY(1,1)	PRIMARY KEY
Username	NVARCHAR(50)	NOT NULL, UNIQUE
Password	NVARCHAR(200)	NOT NULL
CreatedAt	DATETIME	DEFAULT GETDATE()

4.2 dbo.Emp_details

Column	Type	Constraints / Notes
EmpId	NVARCHAR(50)	PRIMARY KEY
EmpName	NVARCHAR(100)	NOT NULL
EmpAge	INT	NOT NULL
EmpContact	NVARCHAR(20)	NULL
EmpGender	NVARCHAR(10)	NULL
CreatedBy	INT NULL	FOREIGN KEY REFERENCES dbo.Users(UserID)

CreatedBy is deliberately nullable: employee rows that predate this feature, or are bulk/migrated in the future, have no known creator and must still be storable and visible.

5. Schema.sql

The complete script (also included as Database/Schema.sql) creates the database and both tables idempotently — safe to run on a clean SQL Server instance, and safe to re-run.

```
IF DB_ID(N'dbCompanyApp') IS NULL CREATE DATABASE [dbCompanyApp];
GO
USE [dbCompanyApp]; GO
IF OBJECT_ID(N'dbo.Users', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Users (
        UserID    INT IDENTITY(1,1) PRIMARY KEY,
        Username  NVARCHAR(50)  NOT NULL UNIQUE,
```

```
        Password NVARCHAR(200) NOT NULL,
        CreatedAt DATETIME DEFAULT GETDATE()
    );
END; GO

IF OBJECT_ID(N'dbo.Emp_details', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Emp_details (
        EmpId NVARCHAR(50) NOT NULL PRIMARY KEY,
        EmpName NVARCHAR(100) NOT NULL,
        EmpAge INT NOT NULL,
        EmpContact NVARCHAR(20) NULL,
        EmpGender NVARCHAR(10) NULL,
        CreatedBy INT NULL,
        CONSTRAINT FK_Emp_CreatedBy FOREIGN KEY (CreatedBy)
            REFERENCES dbo.Users(UserID)
    );
END; GO
```

6. Access Data Migration

db_users.mdb was inspected directly with mdb-tools (mdb-tables / mdb-export), not guessed. It contains one table, tbl_users(username, password), holding **11 real accounts** with no duplicate usernames. Every one of those 11 rows was carried into dbo.Users unchanged, via Database/Migration.sql, guarded by IF NOT EXISTS checks so the script can be re-run safely.

UserID is never inserted manually — it is an IDENTITY column, so SQL Server generates it automatically as each account is migrated. This keeps the migration script decoupled from any assumption about ID ordering or values in the old system.

7. Project Merge Process

EmployeeDetails was used as the host project, per the requirement. Its .csproj was renamed to CompanyApp.csproj, assembly name changed to CompanyApp, target framework raised to 4.8 (it already was), and root namespace left as EmployeeDetails. The three Login-and-Register forms were copied in as complete three-file sets and had their namespaces corrected; Form1 was renamed to frmEmployee. The old Login-and-Register .csproj, .sln, Program.cs, App.config, and Properties folder were not imported.

8. The Three-File Rule

Each WinForms form is really three files: **.cs** (your code), **.Designer.cs** (the other half of the same partial class — field declarations and InitializeComponent()), and **.resx** (embedded resources, wired in via DependentUpon). All three must travel together or the class will not compile; only copying the .cs file removes InitializeComponent() from the type.

9. Namespace Changes

Login_and_Register → EmployeeDetails, applied to both halves of frmLogin, frmRegister and frmDashboard.

10. OleDb to SqlClient Migration

User.cs, frmLogin.cs and frmRegister.cs no longer reference System.Data.OleDb at all. Login and registration now go through User.ValidateLogin / User.UsernameExists / User.RegisterUser, all using SqlConnection/SqlCommand with named parameters. Employee.cs was already on SqlClient with parameters and was extended, not migrated, to add CreatedBy and the LEFT JOIN select. App.config now has exactly one connectionStrings entry, named connString, providerName System.Data.SqlClient.

This also fixes a genuine **SQL-injection vulnerability** in the original login query, which built its WHERE clause by string concatenation and could be bypassed with a password such as ' OR '1'='1. Every query in User.cs and Employee.cs now uses SqlCommand.Parameters.Add(...).

11. User.cs and Session.cs

User.cs implements ValidateLogin (returns the UserID on success, 0 on failure — the ID matters because it feeds Emp_details.CreatedBy), UsernameExists (via ExecuteScalar), and RegisterUser (parameterized INSERT).

Session.cs is a small static class holding the current UserID and Username for the life of the running app, with a Clear() method called on logout.

12. Login Flow

frmLogin.btnLogin_Click validates that both fields are non-empty, calls User.ValidateLogin inside a try/catch, and on a UserID greater than zero sets Session.UserID/Session.Username, shows a new frmDashboard, and hides itself. On failure it shows an error and clears the fields. No SqlConnection is ever held open at the class level; every connection is created inside a using block. Closing frmLogin (FormClosed) calls Application.Exit(), which is what prevents an orphaned hidden process.

13. Registration Flow

frmRegister.btnRegister_Click checks username, password and confirm-password with OR logic (any one empty field fails validation, not only when all three are empty), checks the two passwords match, checks User.UsernameExists before registering, and calls User.RegisterUser on success before returning to the login form.

14. Dashboard

frmDashboard gained a new button, btnManageEmployees ("Manage Employees"), whose Click handler opens frmEmployee modally. Because frmDashboard itself is only reachable after a successful login, there is no path to Employee CRUD that bypasses authentication.

15. Employee CRUD

Form1 was renamed to frmEmployee (class, and the .cs/.Designer.cs/.resx files) with its existing UI and CRUD behavior preserved. All control names were read from the actual Designer file rather than guessed.

16. CreatedBy

Employee.cs gained a nullable property, `public int? CreatedBy { get; set; }`, and the INSERT statement now includes it. In `frmEmployee.btnAdd_Click`, immediately before inserting, `employee.CreatedBy` is set to `Session.UserID`, so every newly added employee is attributed to the logged-in user.

17. LEFT JOIN

The employee SELECT now joins to `dbo.Users` to surface the creator's username:

```
SELECT e.EmpId, e.EmpName, e.EmpAge, e.EmpContact, e.EmpGender,
       e.CreatedBy, u.Username AS CreatedByUsername
FROM   dbo.Emp_details e
LEFT JOIN dbo.Users u ON e.CreatedBy = u.UserID
```

LEFT JOIN, not INNER JOIN, is required specifically because `CreatedBy` is nullable: an INNER JOIN would silently drop any employee row whose `CreatedBy` is NULL (e.g. migrated/legacy rows) from the grid entirely, even though the row still exists in `Emp_details`.

18. Logout

`btnLogout_Click` asks for Yes/No confirmation, then on Yes calls `Session.Clear()`, creates and shows a brand-new `frmLogin`, and closes the current dashboard. It deliberately does not call `Application.Exit()` — that only happens when the (single, freshly created) login form itself is closed, via its `FormClosed` handler.

19. Error Encountered and Solution

While assembling the merged form files, the following real compiler error was reproduced with the Mono C# compiler (`mcs`) — not fabricated:

```
frmLogin.Designer.cs(15,33): error CS0115: `Login_and_Register.frmLogin.Dispose(bool)' is marked
as an override but no suitable method found to override
```

Cause: `frmLogin.cs` had already been edited to namespace `EmployeeDetails`, but `frmLogin.Designer.cs` still said namespace `Login_and_Register`. Since a form's `.cs` and `.Designer.cs` must be two halves of the exact same partial class, the compiler saw two different, unrelated classes instead of one, and the Designer half's `Dispose(bool)` override had nothing valid to override.

Fix: every imported Designer file's namespace line was corrected to `EmployeeDetails` to match its `.cs` half. After that fix, the full project — `Employee.cs`, `User.cs`, `Session.cs`, `Program.cs`, and all four forms — compiled cleanly (0 errors, 0 warnings) against `System.Windows.Forms`, `System.Data`, `System.Drawing` and `System.Configuration`.

20. Testing

This project was assembled and verified in a Linux sandbox with no Visual Studio, MSBuild, .NET Framework targeting pack, or SQL Server instance available. To still get real compiler verification, Mono (`mcs`) plus the `WinForms/Data/Configuration` libraries were installed, and the full merged source was compiled successfully, producing `CompanyApp.exe`. This confirms the C# is syntactically and referentially correct, but is not a substitute for an actual MSBuild/Visual Studio build, and none of the runtime flows (login, register, CRUD, logout) or the SQL scripts were exercised against a live SQL Server instance, since

none exists in this sandbox. The db_users.mdb inspection and data extraction, by contrast, was real and direct, using mdb-tools against the actual uploaded file.

21. GitHub Submission

The repository root contains 24-57685-2_CompanyApp.sln, the CompanyApp project folder, a Database folder with Schema.sql and Migration.sql, README.md, this Report.pdf, and a .gitignore excluding .vs/, bin/, obj/, *.user and *.suo. The old Login-and-Register .sln/.csproj and the .mdb/.accdb files are not part of the merged repository.

22. Conclusion

The two source applications now share one namespace, one data provider, one database, one framework target, and one entry point. The six required conflicts were each traced to a specific line or file before being fixed, the Access accounts were migrated from the real .mdb rather than invented, and the resulting CompanyApp project compiles cleanly. The remaining gap is environment-specific: an actual Visual Studio/MSBuild build and a run against a live SQL Server instance still need to happen outside this sandbox, as documented in the README's verification checklist.

— End of Report —

Student: Sams Ridwan Simran | ID: 24-57685-2 | Faculty: Dr. Iftekhar Mobin | AIUB